Build One Signup Story with XP in Cursor

From customer story to working software with tests leading every step.

Today's story

As a user, I want to sign up for the web application so that I can create an account and log in later.

Thin slice for class: email + password + confirm password, validation, duplicate check, hashing, success/error UI.

Live flow

- Story → criteria → minimal design
- Plan before coding
- Backend TDD first
- API layer next
- Frontend TDD + implementation
- End-to-end check
- Refactor or debug

Goal:
trace one story all the way to working software

Story, scope, and criteria

Story card

Story

As a user, I want to sign up for the web application so that I can create an account and log in later.

Acceptance criteria

- Email must be valid.
- Password must be at least 8 characters.
- Confirm password must match password.
- Duplicate email is rejected.
- Successful signup stores the user with a hashed password.
- UI shows success or error feedback.

Out of scope

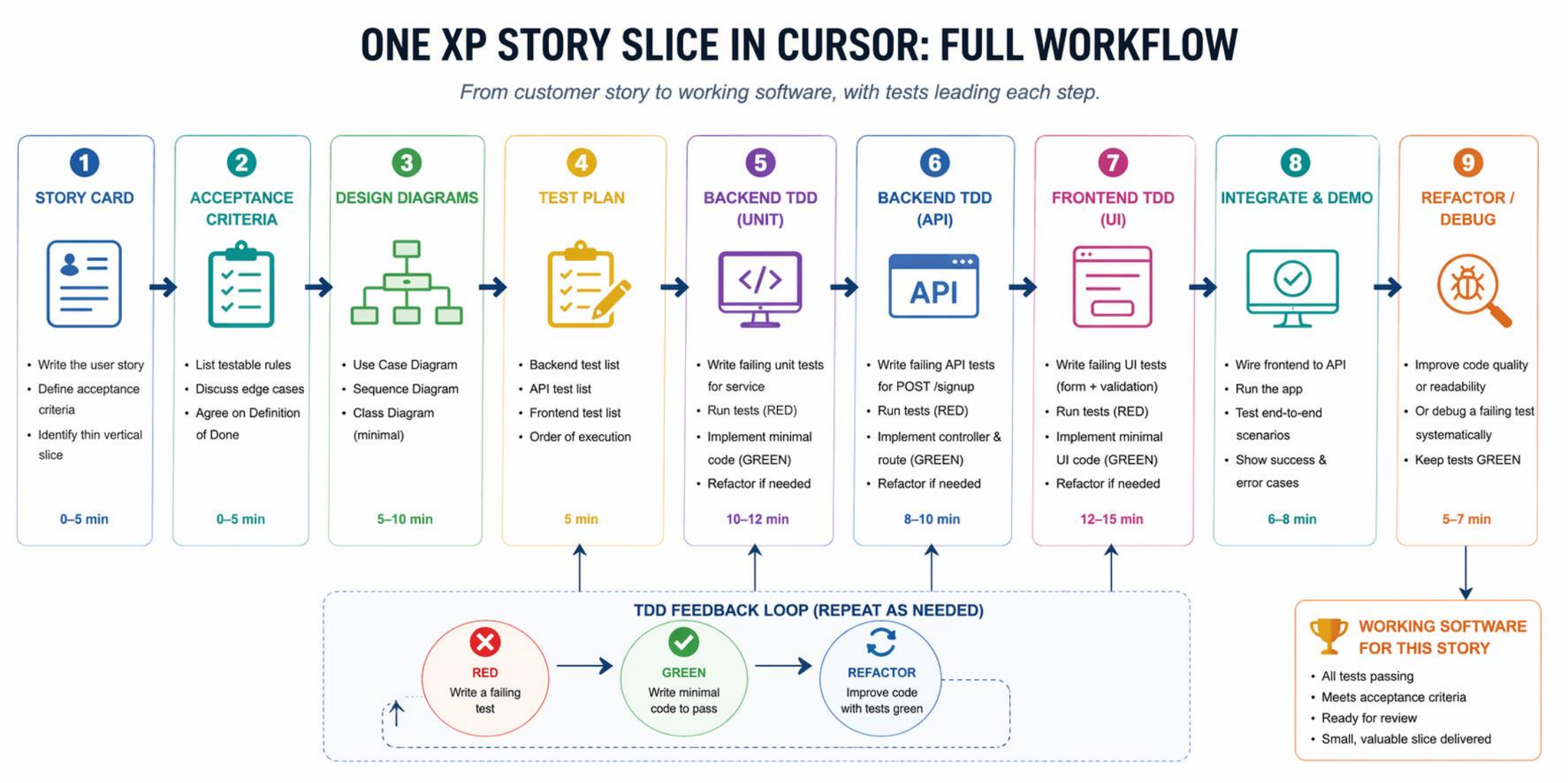
- Login
- Email verification
- Forgot password
- OAuth
- CAPTCHA
- Fancy styling

In scope:

- Signup form with email, password, and confirm password
- Backend POST /api/signup
- Validation on client and server
- Duplicate email check and password hashing
- Success or error feedback in the UI

Tests pass, the feature meets the criteria, and the story is demoable end-to-end.

XP workflow


ONE XP STORY SLICE IN CURSOR: FULL WORKFLOW
From customer story to working software, with tests leading each step.
1 STORY CARD
• Write the user story
• Define acceptance criteria
• Identify thin vertical slice
0–5 min
2 ACCEPTANCE CRITERIA
• List testable rules
• Discuss edge cases
• Agree on Definition of Done
0–5 min
3 DESIGN DIAGRAMS
• Use Case Diagram
• Sequence Diagram
• Class Diagram (minimal)
5–10 min
4 TEST PLAN
• Backend test list
• API test list
• Frontend test list
• Order of execution
5 min
5 BACKEND TDD (UNIT)
• Write failing unit tests for service
• Run tests (RED)
• Implement minimal code (GREEN)
• Refactor if needed
10–12 min
6 BACKEND TDD (API)
API
• Write failing API tests for POST /signup
• Run tests (RED)
• Implement controller & route (GREEN)
• Refactor if needed
8–10 min
7 FRONTEND TDD (UI)
• Write failing UI tests (form + validation)
• Run tests (RED)
• Implement minimal UI code (GREEN)
• Refactor if needed
12–15 min
8 INTEGRATE & DEMO
• Wire frontend to API
• Run the app
• Test end-to-end scenarios
• Show success & error cases
6–8 min
9 REFACTOR / DEBUG
• Improve code quality or readability
• Or debug a failing test systematically
• Keep tests GREEN
5–7 min
TDD FEEDBACK LOOP (REPEAT AS NEEDED)
RED
Write a failing test
GREEN
Write minimal code to pass
REFACTOR
Improve code with tests green
WORKING SOFTWARE FOR THIS STORY
• All tests passing
• Meets acceptance criteria
• Ready for review
• Small, valuable slice delivered

Cursor prompts to use live Part 1

Open with analysis, minimal design, and a reviewable plan.

1. Story analysis

```
Read @docs/stories/STORY-01-signup.md
Extract the smallest vertical slice we can finish in one class.
Do not write code yet.
Return:
1. story summary
2. acceptance criteria
3. backend tasks
4. frontend tasks
```

2. Minimal design

```
Using @docs/stories/STORY-01-signup.md, generate minimal Mermaid content for:
1. use case diagram
2. sequence diagram
3. class diagram

Keep them tiny and relevant to this story only.
Do not generate code.
```

3. Implementation plan

```
Create a minimal implementation plan for STORY-01 signup.

Constraints:
- strict TDD
- backend first
- frontend second
- no extra features
- finishable in one class
```

Cursor prompts to use live Part 2

Backend first: write failing tests, then add only the minimum code.

4. Backend tests first

```
Write the smallest failing tests for @backend/src/services/AuthService.ts

Cover only:
- valid signup
- invalid email
- password too short
- confirm password mismatch
- duplicate email

Do not implement production code yet.
```

5. Backend implementation

```
Implement only enough production code to make the current failing AuthService tests
pass.

Constraints:
- keep code minimal
- no login
- no JWT
- no email verification
```

6. API layer

```
Add the smallest API-level tests for POST /api/signup.
Cover success, validation failure, and duplicate email.
Then implement the minimum controller and route behavior needed to pass.
```

Cursor prompts to use live Part 3

Move to the UI only after the backend slice is working.

7. Frontend tests first

```
Write the smallest failing frontend tests for @frontend/src/components/SignupForm.tsx

Cover only:
- renders email, password, confirm password
- invalid email validation
- short password validation
- confirm password mismatch
- valid submit
- duplicate-email error

Do not implement the component yet.
```

8. Frontend implementation

```
Implement only enough SignupForm and SignupPage code to make the current frontend tests pass.

Constraints:
- minimal styling
- simple form state
- inline error messages
- success message on successful signup
```

Keep the UI plain. The goal is to show traceability from story → tests → code, not to spend time on styling.

Cursor prompts to use live Part 4

Finish with an end-to-end check, then either refactor or debug.

9. End-to-end check

```
Review whether the frontend and backend are correctly wired
together.
If anything small is missing for a basic end-to-end demo,
make only the smallest necessary edits.
```

10. Refactor or debug

```
All tests are green.
Suggest one small safe refactor and apply only that refactor.

or

A signup test is failing after a recent change.
Debug systematically:
1. likely root cause
2. exact code region
3. smallest fix
4. no unrelated edits
```

Demo checklist

- Open the signup page
- Show invalid email feedback
- Show short password feedback
- Show mismatch feedback
- Show successful signup
- Show duplicate email rejection
- Confirm backend and frontend tests pass

Choose refactor OR debug if time gets tight.

Plan

Time	Focus	What to accomplish
0–10	Story + design	Scope the thin slice and create the three tiny diagrams
10–15	Plan	Review the implementation order before coding
15–35	Backend	Write failing tests, then minimum service + API code
35–53	Frontend	Write failing UI tests, then minimal form code
53–60	Finish	End-to-end demo plus refactor or debug

Wrap-up: what students should leave with

One story. One trace. One working slice.

We should see

- A story card becoming precise acceptance criteria
- Minimal diagrams used to communicate one slice
- TDD as red → green → refactor, not “generate everything”
- Backend and frontend connected through tests
- A small, demoable outcome at the end of class

> “Every artifact in this repo traces back to one story card. That is the point of XP: small scope, constant feedback, working software.”

Key reminders

- Constrain every prompt by phase: analyze, design, tests, code, refactor, debug.
- Keep “tests first” and “smallest useful slice.”
- Use Cursor as a pair programmer, not as an autopilot.